LP3 Practical Theory

Design and Analysis of Algorithms (DAA) | Machine Learning (ML) | Blockchain Technology (BT)

Subject	Practicals covered
DAA	Fibonacci, Huffman Encoding, Fractional Knapsack, 0-1 Knapsack, N-Queens
ML	Uber Fare Prediction, Email Spam (KNN/SVM), Bank Churn (ANN), Gradient Descent, K-Means
BT	Bank Account Smart Contract, Student Data Smart Contract (Solidity)

PART 1: DAA (Design and Analysis of Algorithms)

DAA-1: Fibonacci (Recursive and Non-Recursive)

Aim

Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze their time and space complexity.

Theory

The Fibonacci sequence is: 0, 1, 1, 2, 3, 5, 8, 13, 21, ... Each term is the sum of the previous two terms.

Definition: F(n) = F(n-1) + F(n-2), with base cases F(0) = 0 and F(1) = 1.

Non-Recursive (Iterative) Approach

- Uses a loop and keeps only the last two values (a, b). In each iteration: next = a + b; a = b; b = next.
- **Time complexity:** O(n), since the loop runs n times.
- **Space complexity:** O(1), since only a few variables are used.

```
a, b = 0, 1
for i in range(2, n+1):
    a, b = b, a + b
return b
```

Recursive Approach

- The function calls itself: fib(n) = fib(n-1) + fib(n-2). Base case: n <= 1 returns n.
- **Recurrence:** T(n) = T(n-1) + T(n-2) + O(1).
- The same subproblems (e.g. fib(2), fib(3)) are recomputed many times, forming a binary recursion tree of height n.
- **Time complexity:** $O(2^n)$ (tight bound is $O(phi^n)$ where phi = 1.618, the golden ratio).
- **Space complexity:** O(n), because of the maximum depth of the recursion call stack.

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

Optimized: Memoization / Dynamic Programming

Store already computed results in an array or dictionary so each fib(k) is computed once. Time O(n), space O(n). This removes the overlapping-subproblem problem.

Comparison

Method	Time	Space	Remark
Iterative	O(n)	O(1)	Fastest and most memory efficient
Recursive	O(2^n)	O(n)	Very slow for large n, repeated work
Memoization (DP)	O(n)	O(n)	Recursive structure with stored results

Viva Questions

- Why is simple recursion exponential? (Overlapping subproblems.)
- What is the recursion depth for fib(n)? (n.)
- How can Fibonacci be computed in O(log n)? (Matrix exponentiation or fast doubling.)

DAA-2: Huffman Encoding (Greedy)

Aim

Implement Huffman Encoding using a greedy strategy.

Theory

Huffman coding is a **lossless data compression** technique. It assigns **variable-length binary codes** to characters: frequent characters get shorter codes and rare characters get longer codes. It produces **prefix codes**, meaning no code is a prefix of another, so decoding is unambiguous.

Fixed vs variable length: ASCII uses 8 bits per character. Huffman reduces the average number of bits per character depending on frequencies.

Greedy Strategy

At each step, pick the two nodes with the **lowest frequencies** and merge them into a new node. This local optimal choice leads to a globally optimal prefix code.

Algorithm

1. Calculate the frequency of every character.
2. Create a leaf node for each character and insert all into a **min-heap (priority queue)** keyed on frequency.
3. While the heap has more than one node: extract the two nodes with minimum frequency, create a new internal node with frequency equal to their sum (these two become its left and right children), and insert it back.
4. The remaining node is the root of the Huffman tree.
5. Traverse the tree: assign **0** for the left edge and **1** for the right edge. The path from root to a leaf is that character's code.

Example

Character	Frequency	Code (one possible)
a	5	11
b	9	10
c	12	00
d	13	01

(Exact codes may vary with tie-breaking, but the total encoded length stays the same.)

Decoding

Read the encoded bits and walk the tree from the root (0 = left, 1 = right). On reaching a leaf, output the character and restart from the root.

Complexity

- **Time:** O(n log n), because there are n-1 merges and each heap operation costs O(log n).
- **Space:** O(n) for the tree and heap.
- **Compression ratio** = original size / compressed size. Total bits = sum of (frequency x code length).

Viva Questions

- What is a prefix code? Why is it needed?
- Why is Huffman greedy and why is it optimal?
- Which data structure is used? (Min-heap.)
- Is Huffman lossy or lossless? (Lossless.)

DAA-3: Fractional Knapsack (Greedy)

Aim

Solve the fractional knapsack problem using the greedy method.

Theory

Given n items, each with value v_i and weight w_i, and a knapsack of capacity W, choose items to **maximize total value**. In the **fractional** version, items can be broken, so we may take a fraction x_i (0 to 1) of an item.

Objective: maximize sum(v_i x_i) subject to sum(w_i x_i) <= W, where 0 <= x_i <= 1.

Greedy Strategy

Choose items in decreasing order of **value-to-weight ratio** (v_i/w_i).

Algorithm

1. Compute ratio v/w for each item.
2. Sort items in descending order of ratio.
3. Initialize remaining capacity = W and total value = 0.
4. For each item in sorted order: if the whole item fits, take it entirely and reduce capacity; otherwise take the fraction (remaining capacity / weight), add the proportional value, and stop.

Example

Items: (value, weight) = (60,10), (100,20), (120,30); W = 50. Ratios: 6, 5, 4. Take item 1 (10), item 2 (20), then 20/30 of item 3 = 80. Total = 60 + 100 + 80 = **240**.

Complexity

- **Time:** O(n log n) due to sorting (the selection loop is O(n)).
- **Space:** O(1) extra (O(n) if ratios are stored).
- Greedy gives the **optimal** solution for the fractional version, but not for 0-1 knapsack.

Viva Questions

- Why does greedy work here but not in 0-1 knapsack?
- What is the greedy criterion? (Highest value/weight ratio.)

DAA-4: 0-1 Knapsack (Dynamic Programming / Branch and Bound)

Aim

Solve the 0-1 knapsack problem using dynamic programming or branch and bound.

Theory

In the 0-1 knapsack problem each item must be taken **completely or not at all** (x_i is 0 or 1). It exhibits **optimal substructure** and **overlapping subproblems**, so dynamic programming applies.

DP Formulation

Let dp[i][w] be the maximum value using the first i items with capacity w.

- If w_i > w: dp[i][w] = dp[i-1][w] (item cannot be included)
- Otherwise: dp[i][w] = max(dp[i-1][w], v_i + dp[i-1][w - w_i])
- Base: dp[0][w] = 0 and dp[i][0] = 0. Answer = dp[n][W].

To find which items were chosen, backtrack from dp[n][W]: if dp[i][w] != dp[i-1][w], item i was included, so reduce w by w_i.

Complexity

- **Time:** O(n x W).
- **Space:** O(n x W); can be reduced to O(W) with a 1D array (iterating w from W down to w_i).
- It is **pseudo-polynomial** because it depends on the numeric value W, not just input size.

Branch and Bound Alternative

- Builds a state-space tree where each node decides to include or exclude an item.
- Each node has an **upper bound** (usually computed by the fractional knapsack on remaining items). Nodes whose bound is lower than the best solution found so far are **pruned**.
- Uses best-first search with a priority queue. Worst case is exponential, but pruning makes it fast in practice.

Greedy vs DP

Feature	Fractional (Greedy)	0-1 (DP)
Item splitting	Allowed	Not allowed
Method	Greedy by v/w	Dynamic programming
Time	O(n log n)	O(nW)
Optimal?	Yes	Yes (DP); greedy is not

Viva Questions

- Why is 0-1 knapsack NP-hard yet solvable in O(nW)?
- What is the role of the bound in branch and bound?

DAA-5: N-Queens (Backtracking)

Aim

Design an n x n matrix with the first queen already placed, and use backtracking to place the remaining queens.

Theory

Place n queens on an n x n chessboard so that no two queens attack each other: **no two share the same row, column, or diagonal**.

Backtracking is a systematic trial-and-error technique. It builds a solution step by step, and when a partial solution cannot be completed (violates constraints), it **abandons (backtracks)** to the previous step and tries another choice. It is a depth-first search over the state-space tree with pruning.

Algorithm

1. Place the first queen at the given position (row r, column c) and mark it fixed.
2. Process rows one by one (skipping the row of the fixed queen).
3. For the current row, try each column; check **isSafe(row, col)**.
4. If safe, place the queen and recurse to the next row.
5. If the recursive call fails, remove the queen (backtrack) and try the next column.
6. If all rows are filled, a solution is found; print the matrix. If no column works, return false.

isSafe Conditions

- **Column:** no queen already in the same column.
- **Left diagonal:** row - col is the same for cells on the same diagonal.
- **Right diagonal:** row + col is the same for cells on the same anti-diagonal.

Complexity

- **Time:** O(n!) in the worst case (much less in practice thanks to pruning).
- **Space:** O(n) for the column array (plus recursion stack), or O(n^2) when using a full board matrix.
- Number of solutions: 4-Queens has 2, 8-Queens has 92 (12 distinct).

Viva Questions

- What is backtracking? How does it differ from brute force?
- Is a solution always possible with the first queen fixed? (Not always.)
- Why is the time O(n!)?

PART 2: ML (Machine Learning)

Common Concepts

- **Supervised learning:** learns from labeled data (regression: continuous output; classification: discrete output).
- **Unsupervised learning:** finds patterns in unlabeled data (clustering).
- **Train/Test split:** usually 70/30 or 80/20, to check performance on unseen data.
- **Feature scaling:** Standardization (z = (x - mean)/std) or Min-Max scaling. Fit the scaler on training data only, to avoid data leakage.
- **Overfitting:** performs well on training, poorly on test data. **Underfitting:** performs poorly on both.
- **Pre-processing steps:** handle missing values, remove duplicates, encode categorical variables, scale features, treat outliers.

ML-1: Uber Fare Prediction (Linear Regression and Random Forest)

Aim

Predict the price of an Uber ride; pre-process data, identify outliers, check correlation, implement Linear Regression and Random Forest Regression, and compare R2, RMSE, etc.

Dataset

Uber Fares dataset (Kaggle). Columns: key, fare_amount (target), pickup_datetime, pickup_longitude, pickup_latitude, dropoff_longitude, dropoff_latitude, passenger_count.

1. Pre-processing

- Drop unnecessary columns (Unnamed: 0, key). Check and drop null values (df.isnull().sum(), df.dropna()).
- Convert pickup_datetime to datetime and extract **year, month, day, hour, day of week**.
- Remove invalid data: fare_amount <= 0, passenger_count = 0 or very large, latitude outside -90 to 90, longitude outside -180 to 180, zero coordinates.
- Create a **distance** feature from pickup and dropoff coordinates using the **Haversine formula**, which gives the great-circle distance between two points on a sphere:

$a = \sin^2(dlat/2) + \cos(lat1) \times \cos(lat2) \times \sin^2(dlon/2)$; distance = $2R \times \arcsin(\sqrt{a})$, R = 6371 km.

2. Outlier Detection

- An **outlier** is a value significantly different from the other observations (e.g. fare of 500 dollars for 1 km).
- **Box plot:** points beyond the whiskers are outliers.
- **IQR method:** IQR = Q3 - Q1. Outliers are below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR.
- **Z-score:** |z| > 3 is considered an outlier.
- Treatment: remove, cap (winsorize), or transform.

3. Correlation

- **Pearson correlation coefficient (r)** ranges from -1 (perfect negative) to +1 (perfect positive); 0 means no linear relation.

- Computed using df.corr() and visualized as a **heatmap** (seaborn).
- Distance has the strongest correlation with fare_amount. Highly correlated features among themselves show multicollinearity.

4. Models

Linear Regression: models the target as a linear combination of features: y = b0 + b1x1 + b2x2 + ... + bnxn. Coefficients are found by **ordinary least squares**, minimizing the sum of squared residuals. Assumptions: linearity, independence, homoscedasticity, normality of errors, no multicollinearity.

Random Forest Regression: an **ensemble** of many decision trees. Each tree is trained on a **bootstrap sample** (bagging) and uses a random subset of features at each split. The final prediction is the **average** of all trees. It captures non-linear relationships, is robust to outliers, and reduces variance/overfitting. Key hyperparameters: n_estimators, max_depth, min_samples_split.

5. Evaluation Metrics

Metric	Formula	Meaning
R2 score	1 - SS_res / SS_tot	Fraction of variance explained; closer to 1 is better
MSE	mean of (y - y_pred)^2	Average squared error
RMSE	sqrt(MSE)	Error in the same unit as fare
MAE	mean of |y - y_pred|	Average absolute error

Conclusion: Random Forest usually gives higher R2 and lower RMSE than Linear Regression because the relationship between fare, distance, time and location is non-linear.

Viva Questions

- Why use the Haversine formula?
- Difference between bagging and boosting?
- What does R2 = 0.8 mean?
- Why is RMSE more sensitive to outliers than MAE?

ML-2: Email Spam Classification (KNN and SVM)

Aim

Classify emails as Spam or Not Spam using K-Nearest Neighbors and Support Vector Machine and analyze their performance.

Theory

This is a **binary classification** problem: Normal state = Not Spam (0), Abnormal state = Spam (1). The emails.csv dataset contains word-frequency counts of common words for each email, with a label column. The Email No. column is dropped before training.

K-Nearest Neighbors (KNN)

- A **lazy, instance-based, non-parametric** algorithm: there is no training phase; it stores the data.
- To classify a new point: compute distance to all training points (Euclidean: sqrt(sum $(x_i - y_i)^2$); also Manhattan, Minkowski), select the **k nearest**, and take a **majority vote**.

- Small k: sensitive to noise (overfitting). Large k: smoother boundary (underfitting). Choose k by cross-validation, usually odd.
- Needs **feature scaling**. Prediction is slow on large data, and it suffers in high dimensions (curse of dimensionality).

Support Vector Machine (SVM)

- Finds the **optimal separating hyperplane** that **maximizes the margin** between the two classes. The nearest points to the hyperplane are the **support vectors**.
- **Hard margin** for perfectly separable data; **soft margin** allows some misclassification, controlled by parameter **C** (large C = less tolerance, small margin; small C = wider margin).
- **Kernel trick** maps data into higher dimensions to handle non-linear boundaries: Linear, Polynomial, RBF (Gaussian), Sigmoid. **gamma** controls the influence of a single point in RBF.
- Effective in high-dimensional spaces such as text data.

Performance Analysis

Use the **confusion matrix**:

	Predicted Not Spam	Predicted Spam
Actual Not Spam	TN	FP
Actual Spam	FN	TP

- Accuracy = (TP + TN) / (TP + TN + FP + FN)
- Precision = TP / (TP + FP) (of predicted spam, how many are really spam)
- Recall = TP / (TP + FN) (of real spam, how many were caught)
- F1-score = 2 x Precision x Recall / (Precision + Recall)

In spam detection, a **false positive** (a genuine email marked spam) is costly, so precision matters. Usually SVM performs better than KNN on this dataset and predicts faster.

Viva Questions

- Why is KNN called a lazy learner?
- What are support vectors?
- What is the kernel trick?
- Accuracy vs precision vs recall: which matters for spam?

ML-3: Bank Customer Churn Prediction (Neural Network)

Aim

Build a neural-network classifier to predict whether a bank customer will leave (churn) in the next 6 months.

Dataset

Churn_Modelling.csv: 10,000 rows, 14 columns (RowNumber, CustomerId, Surname, CreditScore, Geography, Gender, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary, **Exited** = target).

Neural Network Theory

- An **Artificial Neural Network (ANN)** consists of an **input layer**, one or more **hidden layers**, and an **output layer** of connected neurons.
- Each neuron computes: output = activation(sum(w x input) + bias).
- **Activation functions: ReLU** f(x) = max(0, x) in hidden layers (fast, avoids vanishing gradient); **Sigmoid** $f(x) = 1/(1 + e^{-x})$ in the output layer for binary classification (gives probability between 0 and 1); Tanh and Softmax are others.
- **Forward propagation:** inputs pass through the layers to produce a prediction.
- **Loss function:** Binary Cross-Entropy = -[y log(p) + (1-y) log(1-p)].
- **Backpropagation:** computes gradients of the loss w.r.t. weights using the chain rule; an **optimizer** (SGD, Adam) updates the weights.
- **Epoch:** one full pass over the training data. **Batch size:** samples per weight update.

Steps

1. **Read the dataset** with pandas.
2. **Pre-process:** drop RowNumber, CustomerId, Surname (not useful); encode Geography (one-hot) and Gender (label encoding).
3. **Separate features X and target y** (Exited); split into train and test sets (e.g. 80/20) with train_test_split.
4. **Normalize** using StandardScaler: fit_transform on train, transform on test.
5. **Build the model** (Keras Sequential): Dense(hidden, relu) layers and Dense(1, sigmoid) output; compile with optimizer=adam, loss=binary_crossentropy, metrics=accuracy; fit with epochs and batch size.
6. **Predict** and convert probabilities to classes with threshold 0.5.
7. **Evaluate:** print accuracy_score and confusion_matrix.

Points of Improvement

- Tune number of hidden layers/neurons, epochs, batch size, learning rate.
- Add **Dropout** or L2 regularization to reduce overfitting; use **Early Stopping**.
- Handle **class imbalance** (about 20% churners) with SMOTE, oversampling, or class weights.
- Try other optimizers and feature engineering; use cross-validation and adjust the decision threshold.

Typical accuracy is about 85 to 86%. Because of class imbalance, also look at precision, recall, and F1.

Viva Questions

- Why use sigmoid in the output layer?
- Why fit the scaler only on training data?
- What is the role of backpropagation?
- How do you reduce overfitting in a neural network?

ML-4: Gradient Descent

Aim

Implement the Gradient Descent algorithm to find the local minimum of a function, e.g. $y = (x + 3)^2$ starting from x = 2.

Theory

Gradient Descent is a first-order **iterative optimization** algorithm that finds a minimum of a differentiable function by repeatedly moving in the direction of the **negative gradient** (steepest descent).

Update rule: $x_{new} = x_{old}$ - alpha x $f'(x_{old})$, where alpha is the **learning rate**.

Worked Example

$f(x) = (x + 3)^2$, so $f'(x) = 2(x + 3)$. Start x = 2, alpha = 0.1.

Iteration	x	f'(x) = 2(x+3)	New x
1	2	10	2 - 0.1(10) = 1
2	1	8	1 - 0.8 = 0.2
3	0.2	6.4	0.2 - 0.64 = -0.44
...	...	...	converges to -3

The minimum is at **x = -3**, where y = 0.

Algorithm

1. Initialize x, learning rate alpha, tolerance, and max iterations.
2. Compute gradient f'(x).
3. Update x = x - alpha x f'(x).
4. Stop when |change in x| < tolerance or max iterations reached.

Key Points

- **Large alpha:** may overshoot or diverge. **Small alpha:** very slow convergence.
- Convex functions: converges to the global minimum. Non-convex: may get stuck in a local minimum or saddle point.
- Variants: Batch GD, Stochastic GD (one sample), Mini-batch GD.
- Gradient Ascent is the opposite: it moves along the positive gradient to find a maximum.

Viva Questions

- What is the learning rate?
- Why negative gradient?
- Difference between batch, stochastic and mini-batch GD?

ML-5: K-Means Clustering (Sales Data)

Aim

Implement K-Means (or hierarchical) clustering on sales_data_sample.csv and determine the number of clusters using the elbow method.

Theory

K-Means is an **unsupervised** partition-based clustering algorithm that groups data into K clusters such that points in a cluster are similar and points in different clusters are dissimilar. It minimizes **WCSS** (Within-Cluster Sum of Squares), also called inertia.

Algorithm

1. Choose the number of clusters K.
2. Initialize K centroids (randomly or with k-means++).
3. **Assignment step:** assign each point to the nearest centroid (Euclidean distance).
4. **Update step:** recompute each centroid as the mean of its points.
5. Repeat steps 3 and 4 until centroids no longer change (convergence) or max iterations.

Elbow Method

- Run K-Means for K = 1, 2, ..., 10 and compute WCSS for each.
- Plot K against WCSS. WCSS decreases as K increases; the point where the curve bends like an elbow (rate of decrease slows) is the optimal K.
- The **silhouette score** (-1 to 1) can also be used to verify the choice.

Pre-processing for sales_data_sample.csv

- Read the file with encoding='latin1'. Select numeric columns (e.g. QUANTITYORDERED, PRICEEACH, SALES, MSRP).
- Drop irrelevant columns and null values; scale the features with StandardScaler (K-Means is distance based).

Hierarchical Clustering (Alternative)

- **Agglomerative (bottom-up):** each point starts as its own cluster; the two closest clusters are merged repeatedly.
- **Divisive (top-down):** start with one cluster and split.
- Linkage types: single, complete, average, Ward. Result is shown as a **dendrogram**; cutting it at a chosen height gives the clusters.
- No need to specify K in advance, but it is $O(n^2)$ or worse, so it is slower on large data.

Limitations of K-Means

- Must choose K in advance; sensitive to initial centroids and outliers; assumes spherical clusters of similar size.

Viva Questions

- What is WCSS?
- How do you choose K?
- K-Means vs hierarchical clustering?
- Why scale features?

PART 3: BT (Blockchain Technology)

Core Concepts

- **Blockchain:** a distributed, decentralized, **immutable** ledger of transactions grouped in blocks. Each block holds transactions, a timestamp, its own hash, and the **previous block's hash**, forming a chain.
- **Consensus mechanisms:** Proof of Work (miners solve puzzles), Proof of Stake (validators stake coins; used by Ethereum since 2022).
- **Ethereum:** a blockchain platform with a programmable layer. Its native currency is **Ether (ETH)**.
- **Smart contract:** self-executing program stored on the blockchain that runs automatically when conditions are met; it is immutable once deployed and transparent.
- **Solidity:** a statically-typed, contract-oriented language for Ethereum smart contracts. Compiled to bytecode and run by the **EVM** (Ethereum Virtual Machine).
- **ABI:** the interface description used to call contract functions.
- **Wallet / account:** MetaMask holds keys. **EOA** (externally owned account) vs contract account.
- **Test networks:** Sepolia (recommended; Goerli is deprecated), Holesky. Test ETH comes free from a **faucet**.
- **Remix IDE:** a browser-based IDE to write, compile, deploy and test Solidity contracts (environments: Remix VM, Injected Provider - MetaMask).

Gas and Transaction Fee

- **Gas:** the unit measuring computational effort. Each EVM operation costs gas; it prevents infinite loops and spam.
- **Gas limit:** maximum gas you are willing to use. **Gas used:** actual gas consumed.
- **Gas price** is measured in **Gwei** (1 Gwei = 10^{-9} ETH).
- **Transaction fee = Gas used x Gas price.** After EIP-1559: fee = Gas used x (Base fee + Priority fee).
- Deployment costs much more gas than a function call. Writing to storage is costly; reading with **view/pure** functions called off-chain is free.

Solidity Basics

- **Data types:** uint, int, bool, address, string, bytes, mapping, array, struct, enum.
- **Visibility:** public, private, internal, external. **State mutability:** view (reads state), pure (no state access), payable (can receive ETH).
- **Global variables:** msg.sender (caller), msg.value (ETH sent in wei), block.timestamp.
- **require(condition, message):** reverts if the condition fails. **Events:** log data on the blockchain (emit).
- **Constructor:** runs once at deployment.

BT-1: Bank Account Smart Contract

Aim

Write a smart contract on a test network for a customer's bank account with Deposit, Withdraw and Show Balance operations.

Theory

- A **mapping(address => uint) balances** stores each customer's balance keyed by their address.
- **deposit():** a **payable** function. ETH sent with the call (msg.value) is added to balances[msg.sender].
- **withdraw(amount):** checks with require(balances[msg.sender] >= amount), reduces the balance first, then sends ETH using payable(msg.sender).transfer(amount). Updating state before sending prevents **re-entrancy** attacks (checks-effects-interactions pattern).
- **getBalance():** a **view** function returning balances[msg.sender] (free to call).
- Deposit and withdraw change state, so they are transactions that cost gas.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Bank {
    mapping(address => uint) private balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }
    function withdraw(uint amount) public {
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
        payable(msg.sender).transfer(amount);
    }
    function showBalance() public view returns (uint) {
        return balances[msg.sender];
    }
}
```

Steps to Execute

1. Open Remix IDE, create Bank.sol and paste the code.
2. Compile with the Solidity compiler.
3. In Deploy and Run, choose environment **Injected Provider - MetaMask** connected to **Sepolia** (or Remix VM for a local test).
4. Deploy and confirm in MetaMask.
5. Enter a value (e.g. 1 Ether or in wei) and call **deposit**; call **withdraw**; call **showBalance**.
6. Note the transactions, gas used, and fee in the Remix console or on Etherscan.

Viva Questions

- What is msg.sender and msg.value?
- Why mark deposit as payable?
- What is a re-entrancy attack?
- Difference between transfer, send and call?

BT-2: Student Data Contract (Structure, Array, Fallback)

Aim

Create student data using structures, arrays and a fallback function; deploy on Ethereum and observe the transaction fee and gas values.

Theory

- **Structure (struct):** a user-defined type grouping related fields, e.g. struct Student { uint id; string name; uint marks; }.
- **Array:** a collection of elements. A **dynamic array** Student[] can grow with push(); fixed-size arrays have a set length. Elements are accessed by index, and length gives the size.
- **Fallback function:** fallback() external payable {} is executed when a call is made to a function that does not exist, or when ETH is sent with data and no receive() function exists. **receive() external payable** handles plain ETH transfers with empty calldata. They let the contract handle unexpected calls and accept ETH.
- Gas is higher when storing strings and growing arrays because storage writes are expensive (about 20,000 gas for a new storage slot).

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentData {
    struct Student {
        uint id;
        string name;
        uint marks;
    }
    Student[] public students;

    function addStudent(uint _id, string memory _name, uint _marks) public {
        students.push(Student(_id, _name, _marks));
    }
    function getStudent(uint index) public view returns (uint, string memory, uint) {
        Student memory s = students[index];
        return (s.id, s.name, s.marks);
    }
    function getCount() public view returns (uint) {
        return students.length;
    }
    fallback() external payable { }
    receive() external payable { }
}
```

Observing Gas and Transaction Fee

- Deploy using Remix with MetaMask on Sepolia. In the Remix console, expand the transaction to see **gas, transaction cost, execution cost, gas price**.
- On **Sepolia Etherscan**, search the transaction hash to see Gas Limit and Usage, Gas Price (Gwei), and Transaction Fee.
- Compare: **deployment** (highest gas), **addStudent** (moderate; storage write), **getStudent / getCount** (no gas when called externally, as they are view functions), and the fallback call.
- Example calculation: gas used 100,000 x gas price 20 Gwei = 2,000,000 Gwei = 0.002 ETH.

Viva Questions

- What is the difference between fallback and receive?
- Why is deployment gas high?

- What is the difference between storage, memory and calldata?
- What is an array vs a mapping in Solidity?

Last-Minute Revision Table

Practical	Key idea	Complexity / Metric
Fibonacci	Recursion vs iteration	Rec: O(2^n) time, O(n) space; Iter: O(n), O(1)
Huffman	Greedy, min-heap, prefix code	O(n log n)
Fractional Knapsack	Greedy by v/w	O(n log n)
0-1 Knapsack	DP table	O(nW)
N-Queens	Backtracking	O(n!)
Uber fare	Haversine, IQR, LR vs RF	R2, RMSE, MAE
Spam	KNN vs SVM	Accuracy, precision, recall, F1
Churn	ANN, ReLU, sigmoid, BCE	Accuracy, confusion matrix
Gradient Descent	x = x - alpha f'(x)	Converges to x = -3
K-Means	Centroids, elbow	WCSS
Bank contract	payable, mapping, require	Fee = gas used x gas price
Student contract	struct, array, fallback	Gas on deploy vs call

All the best for your practical!